Midterm 2

Put your name here

DATA 412/612

2026-11-11

November 11, 2026. You have 75 minutes. Steps 1 to 13 are 100 points. Step 14 is 10 points of extra credit.

Put your name in the subtitle at the top of this file. Follow the steps in order. Write the code for each step in the chunk under that step. Render to PDF. Submit this `.qmd` and the PDF before the 75 minutes are up.

The goal is a plot of mean potential against the standard deviation of sprint speed. The players in that plot have a simple name, a first name that starts with M, and a family name of at least 8 letters. A simple name has exactly two uppercase letters and exactly one space.

- You may use your notes and the R help files inside RStudio.
- You may use this course website, https://amousavi1.github.io/612/data-612.html, and David Gerard's data page for the database file, https://dcgerard.github.io/stat_412_612/data.html.
- You may not use Google, any other website, or an LLM.
- You may not communicate with anyone else.
- Use the tidyverse where a tidyverse verb exists. For text, use stringr. Step 14 is a function; base R is appropriate for the `stop()` check.
- `collect()` before `separate()`, `unite()`, or any `str_` verb. Those run in R. This SQLite file does not run a regular expression.
- Either pipe is fine: `|>` or `%>%`.
- Partial credit is given for code that runs. If you are stuck, go to the next step.

Each step starts from the data frame the previous step made. Step 8 prints a summary and must not drop any rows. Steps 9 and 10 continue from the data frame in step 7.

1. Load the packages (5 points)

Load tidyverse, DBI, RSQLite, and dbplyr. This chunk must not display messages or warnings.

2. Connect to the soccer database (8 points)

`soccer.sqlite` is the European soccer database on David Gerard's data page, https://dcgerard.github.io/stat_412_612/data.html. Download it, unzip it, and save `soccer.sqlite` in a folder `data` next to this file.

Connect with `dbConnect()` and `SQLite()`, using the relative path `data/soccer.sqlite`. Assign the connection to `con`.

Print the table names. Print how many tables remain when `sqlite_sequence` is not counted.

3. Bring two tables into R (8 points)

Collect `Player` into a data frame. Collect `Player_Attributes` into a data frame, and keep only `player_api_id`, `potential`, and `sprint_speed`.

From step 4 on, work with these data frames.

4. Prove that `player_api_id` is a key (10 points)

Prove that `player_api_id` is a primary key of `Player`, and that it is not a primary key of `Player_Attributes`.

For each data frame, print the number of rows and the number of distinct `player_api_id`. In one sentence, say what those two comparisons show.

5. Keep the name columns (5 points)

From `Player`, keep `player_api_id` and `player_name`.

6. Keep the simple names (8 points)

A simple name has exactly two uppercase letters and exactly one space. Keep those rows. Use `str_count()` for both counts.

7. Split each simple name (6 points)

Split `player_name` on the space into `first_name` and `family_name`, with `separate()`.

8. Initials (8 points)

From the data frame in step 7, build `initials` with `str_sub()` and `str_c()`. Take the first letter of `first_name`, a period, the first letter of `family_name`, and a period. `Luis Martinez` becomes `L.M.`.

Show the five initials that occur most often, with their counts, largest count first. Break a tie by the initial, A to Z.

Leave the data frame from step 7 intact for the next step.

9. Keep the players in the plot (8 points)

From the data frame in step 7, keep rows whose `first_name` starts with `M` and whose `family_name` has at least 8 characters. Use `str_starts()` and `str_length()`.

10. Put the name back together (6 points)

Unite `first_name` and `family_name` into `player_name`, with one space between them. Keep `player_api_id` and `player_name` only. Assign the result to `df1`.

Print how many rows `df1` has.

11. Average potential and the spread of sprint speed (10 points)

From the `Player_Attributes` data frame, group by `player_api_id`. For each player, compute `potential_avg`, the mean of `potential`, and `sd_speed`, the standard deviation of `sprint_speed`. Ignore missing values. Assign the result to `df2`.

12. Join the names to the summary (8 points)

Left join `df1` to `df2` by `player_api_id`. Drop a row when `potential_avg` or `sd_speed` is missing. Print how many rows remain.

13. The plot (10 points)

Plot `sd_speed` on the horizontal axis and `potential_avg` on the vertical axis.

- Points, with an alpha so overlapping points stay visible.
- One straight smooth in black, dashed, with no gray band.
- Use `theme_bw()`. Title the plot `Potential and sprint speed`. Label the axes `SD of sprint speed` and `Mean potential`.

In one sentence, say how mean potential changes as the standard deviation of sprint speed increases.

14. A function (10 extra credit)

Write a function of one argument, `name`.

- If `name` is not a character vector of length 1, or it is missing, stop with the message `name must be one string`.
- Collapse the whitespace with `str_squish()`.
- If a comma appears, drop the comma and everything after it with `str_remove()`, then squish again.
- If the result contains no space, return it unchanged.
- Otherwise return the last word, with `str_extract()` and the pattern `\\S+$`.

Write the function in this chunk, then run it on these four inputs. This chunk already has `#| error: true`, so the last call can stop and the document still renders. Leave that error visible.

- `"  Luis   Martinez "`
- `"Abdoulaye Diallo Sadio,22"`
- `"Luisinho"`
- `c("Aaron", "Taylor")`